Introduction to zk-SNARKs. R1CS

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 What is zk-SNARK?

2 Boolean Circuits

3 Arithmetic Circuits

4 Linear Algebra Recap

5 Rank-1 Constraint System

What is zk-SNARK?

What Is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

What Is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.

What Is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.
- **Succinctness** — the proof size and verification time is relatively small to the computation size and typically does not depend on the size of the data or statement.

What Is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.
- **Succinctness** — the proof size and verification time is relatively small to the computation size and typically does not depend on the size of the data or statement.
- **Non-interactiveness** — to produce the proof, the prover does not need any interaction with the verifier.

What Is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be "extracted".
- **Succinctness** — the proof size and verification time is relatively small to the computation size and typically does not depend on the size of the data or statement.
- **Non-interactiveness** — to produce the proof, the prover does not need any interaction with the verifier.
- **Zero-Knowledge** — the verifier learns nothing about the data used to produce the proof, despite knowing that this data resolves the given problem and that the prover possesses it.

Still don’t get who is SNARK...

Well... Let’s take a look at some example.

Still don’t get who is SNARK...

Well... Let’s take a look at some example.



Imagine you’re part of a treasure hunt...

Still don’t get who is SNARK...

Well... Let’s take a look at some example.



Imagine you’re part of a treasure hunt...

...and you’ve found a hidden treasure chest...



Still don't get who is SNARK...

Well... Let's take a look at some example.



Imagine you're part of a treasure hunt...

...and you've found a hidden treasure chest...



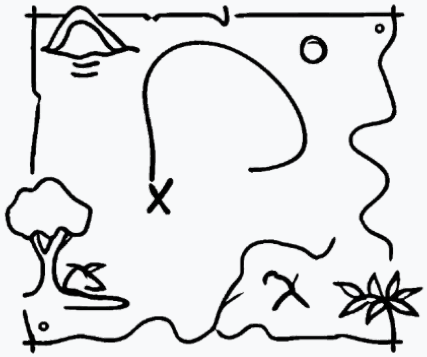


...but how to prove that without revealing the chest location?

Still don't get who is SNARK...

The Problem

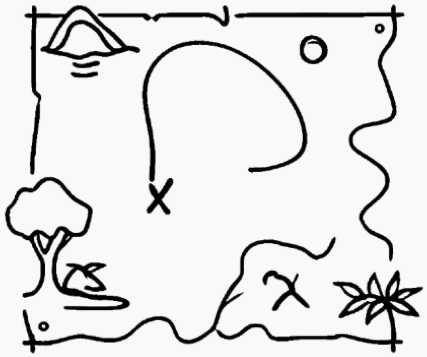
You have found a hidden treasure chest, and you want to prove to the organizer that you know its location without actually revealing that.



Still don't get who is SNARK...

The Problem

You have found a hidden treasure chest, and you want to prove to the organizer that you know its location without actually revealing that.



We can retrieve some information from that:

The Secret Data: the exact treasure location.

The Prover: you.

The Verifier: the treasure hunt organizer.

Ohh... Got it!

Here is how we can apply the zk-SNARK to our problem:

- **Argument of Knowledge**: You need to create a proof that demonstrates you know where the chest is.

Ohh... Got it!

Here is how we can apply the zk-SNARK to our problem:

- **Argument of Knowledge**: You need to create a proof that demonstrates you know where the chest is.
- **Succinct**: The proof you provide is very small and concise. It doesn't matter how large the treasure map is or how many steps it took you to find the chest.

Ohh... Got it!

Here is how we can apply the zk-SNARK to our problem:

- **Argument of Knowledge**: You need to create a proof that demonstrates you know where the chest is.
- **Succinct**: The proof you provide is very small and concise. It doesn't matter how large the treasure map is or how many steps it took you to find the chest.
- **Non-interactive**: You don't need to have a back-and-forth conversation with the organizer to create this proof.

Ohh... Got it!

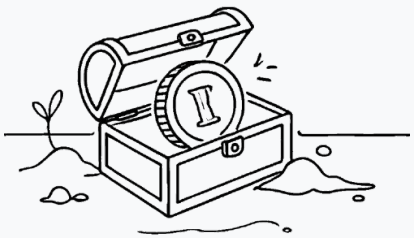
Here is how we can apply the zk-SNARK to our problem:

- **Argument of Knowledge**: You need to create a proof that demonstrates you know where the chest is.
- **Succinct**: The proof you provide is very small and concise. It doesn't matter how large the treasure map is or how many steps it took you to find the chest.
- **Non-interactive**: You don't need to have a back-and-forth conversation with the organizer to create this proof.
- **Zero-Knowledge**: The proof doesn't reveal any information about the actual location of the treasure chest.

Ohh... Got it!

Here is how we can apply the zk-SNARK to our problem:

- **Argument of Knowledge**: You need to create a proof that demonstrates you know where the chest is.
- **Succinct**: The proof you provide is very small and concise. It doesn't matter how large the treasure map is or how many steps it took you to find the chest.
- **Non-interactive**: You don't need to have a back-and-forth conversation with the organizer to create this proof.
- **Zero-Knowledge**: The proof doesn't reveal any information about the actual location of the treasure chest.



Well... The golden coin where the pirates' sign is engraved is our zk-SNARK proof!

The First Question To Resolve

But the problems that we usually want to solve are in a slightly different format.

The First Question To Resolve

But the problems that we usually want to solve are in a slightly different format.

When we need to prove that some element is in a merkle tree, we can’t come to a verifier and give them a "coin"...

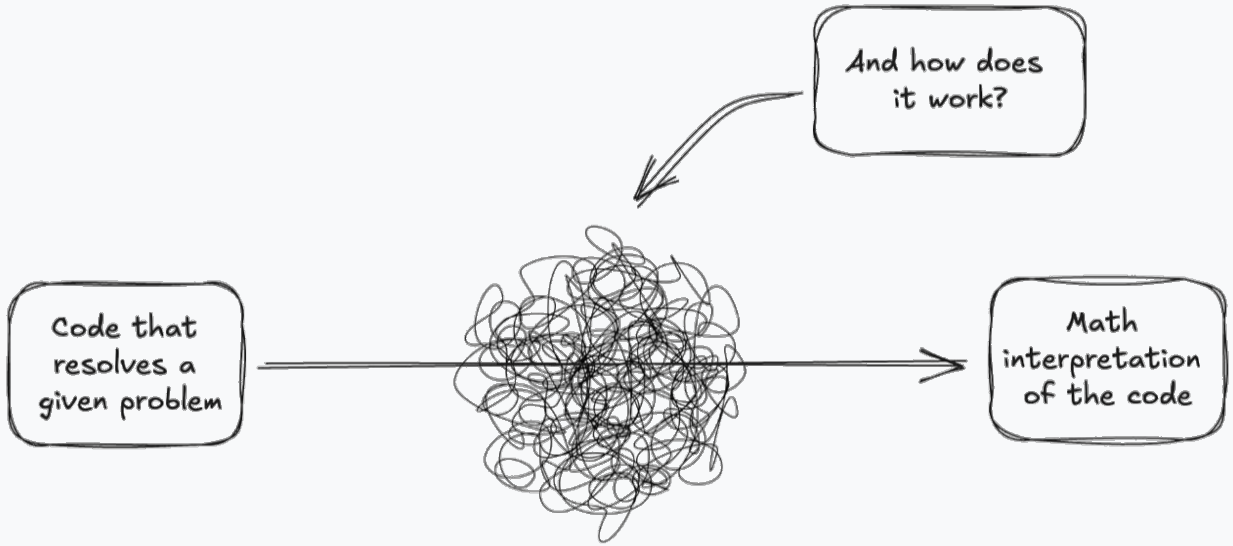
The First Question To Resolve

But the problems that we usually want to solve are in a slightly different format.

When we need to prove that some element is in a merkle tree, we can't come to a verifier and give them a "coin"...

Question

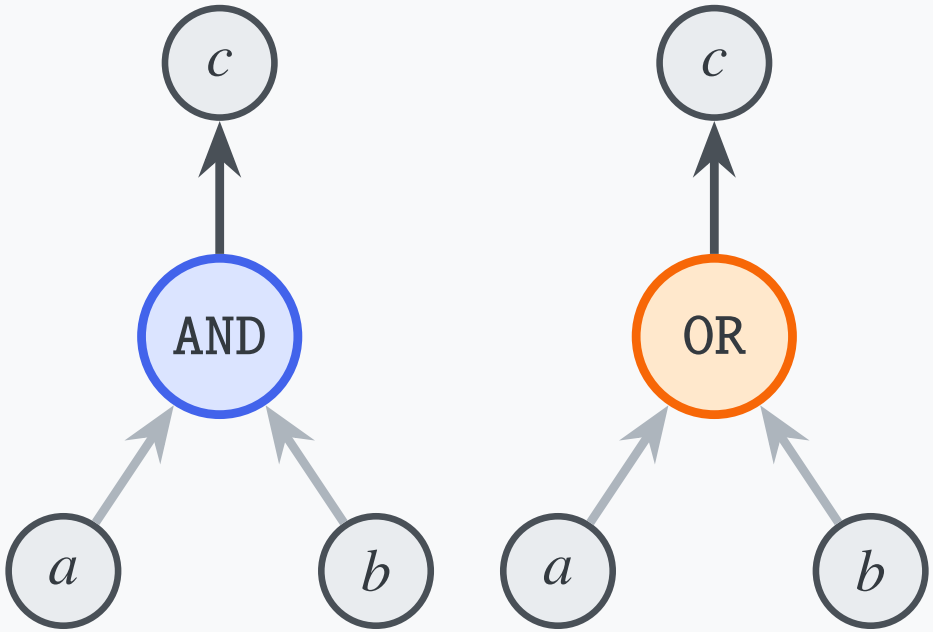
How do we convert a program into a mathematical language?



Boolean Circuits

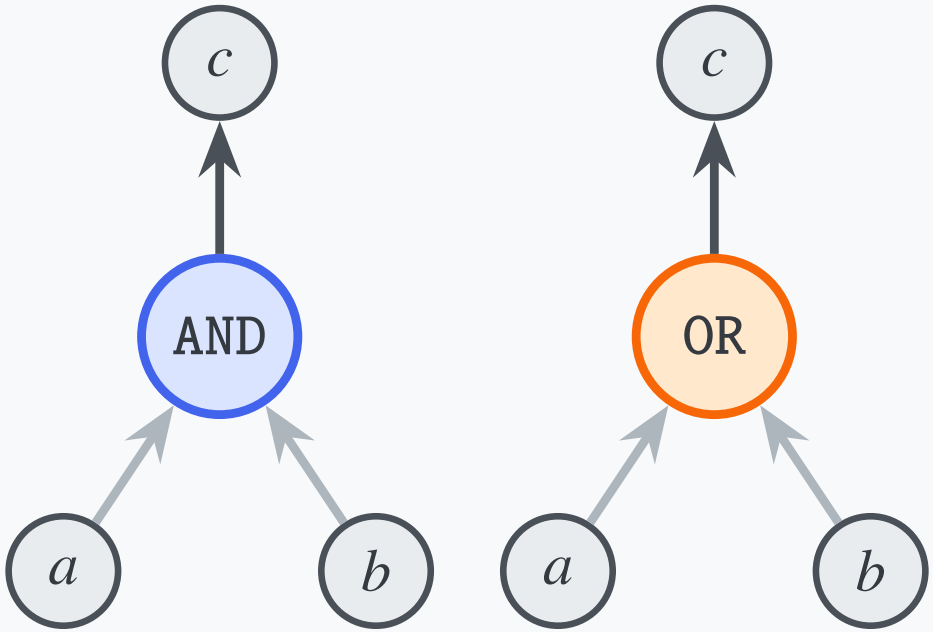
Boolean Circuits

We can do that in a way like the computer does it — **Boolean Circuits**.



Boolean Circuits

We can do that in a way like the computer does it — **Boolean Circuits**.



A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Note

With any of {`AND`, `NOT`} or {`OR`, `NOT`} gates sets one can build any possible logical circuit, they are called **functionally complete** sets.

Boolean Circuit Example

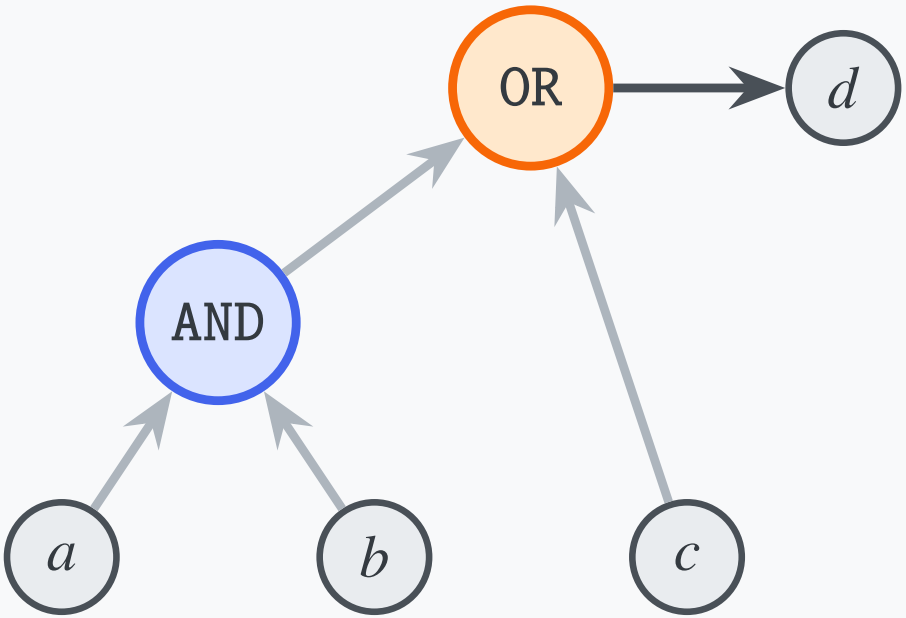


Figure: Example of a circuit evaluating $d = (a \texttt{ AND } b) \texttt{ OR } c$.

Boolean Circuit Example

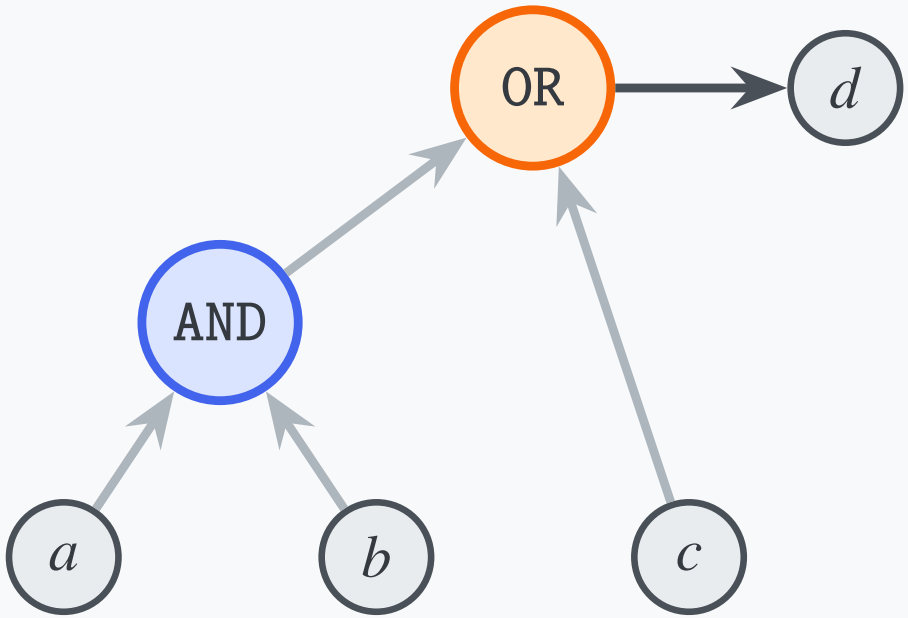


Figure: Example of a circuit evaluating $d = (a \texttt{ AND } b) \texttt{ OR } c$.

Boolean circuits receive an input vector of $\{0, 1\}$ and resolve to true (1) or false (0);

The above circuit can be satisfied with the next values:

$$a = 1, \quad b = 1, \quad c = 0, \quad d = 1$$

SHA-256 Boolean circuit

File	No. ANDs	No. XORs	No. INVs
sha256Final.txt	22,272	91,780	2,194

Figure: Stats of a SHA256 boolean circuit implementation.

More than 100000 gates. Impressive, isn't it?

But it also shows how inconvenient the boolean circuits are.

Arithmetic Circuits

Arithmetic Circuits

Similar to Boolean Circuits, the **Arithmetic Circuits** consist of gates and wires.

- **Wires**: elements of some finite field $\mathbb{F}$.
- **Gates**: field addition ($+$) and multiplication ($\times$).

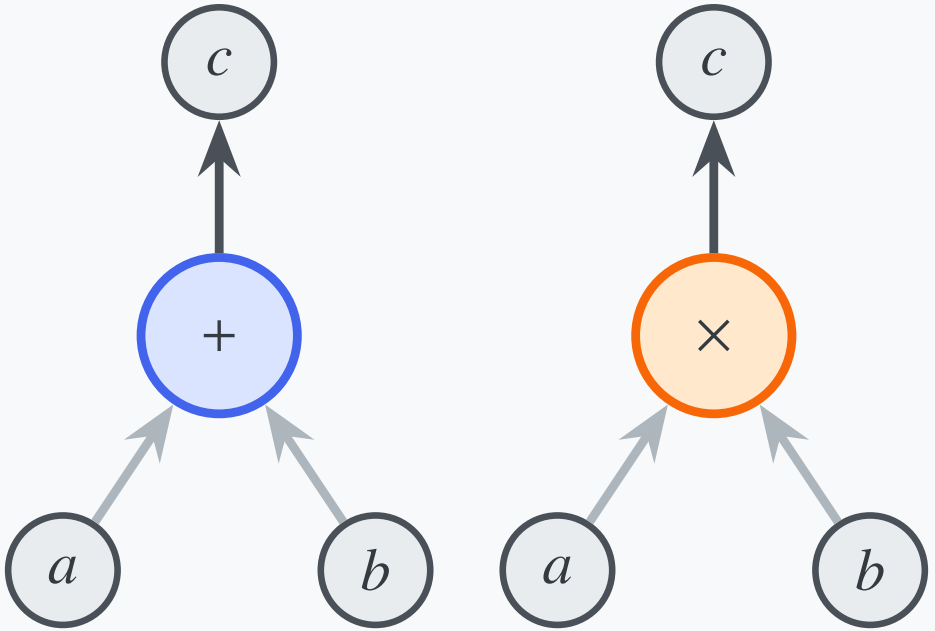


Figure: Addition and Multiplication Gates

Arithmetic Circuits Example I

Example

```
def multiply(a: F, b: F) -> F:
    return a * b
```

Arithmetic Circuits Example I

Example

```
def multiply(a: F, b: F) -> F:
    return a * b
```

This can be represented as a circuit with only one (multiplication) gate:

$$r = a \times b$$

Arithmetic Circuits Example I

Example

```
def multiply(a: F, b: F) -> F:
    return a * b
```

This can be represented as a circuit with only one (multiplication) gate:

$$r = a \times b$$

The witness vector (essentially, our solution vector) is $\mathbf{z} = (r, a, b)$, for example: $(6, 2, 3)$.

We assume that the a and b are input values.

Arithmetic Circuits Example I

Example

```
def multiply(a: F, b: F) -> F:
    return a * b
```

This can be represented as a circuit with only one (multiplication) gate:

$$r = a \times b$$

The witness vector (essentially, our solution vector) is $\mathbf{z} = (r, a, b)$, for example: $(6, 2, 3)$.

We assume that the a and b are input values.

Note

We can think of the "=" in the gate as an assertion.

Arithmetic Circuits Example II

Example

Now, suppose we want to implement the evaluation of the polynomial $Q(x_1, x_2) = x_1^3 + x_2^2 \in \mathbb{F}[x_1, x_2]$ using arithmetic circuits.

```
def evaluate(x1: F, x2: F) -> F:
    return x1**3 + x2**2
```

Looks easy, right?

Arithmetic Circuits Example II

Example

Now, suppose we want to implement the evaluation of the polynomial $Q(x_1, x_2) = x_1^3 + x_2^2 \in \mathbb{F}[x_1, x_2]$ using arithmetic circuits.

```
def evaluate(x1: F, x2: F) -> F:
    return x1**3 + x2**2
```

Looks easy, right? But the circuit is now much less trivial.

$$\begin{aligned} x_1^2 &= x_1 \times x_1 \\ x_1^3 &= x_1^2 \times x_1 \\ x_2^2 &= x_2 \times x_2 \\ Q &= x_1^3 + x_2^2 \end{aligned} \quad \text{or} \quad \begin{aligned} r_1 &= x_1 \times x_1 \\ r_2 &= r_1 \times x_1 \\ r_3 &= x_2 \times x_2 \\ Q &= r_2 + r_3 \end{aligned}$$

Arithmetic Circuits Example II

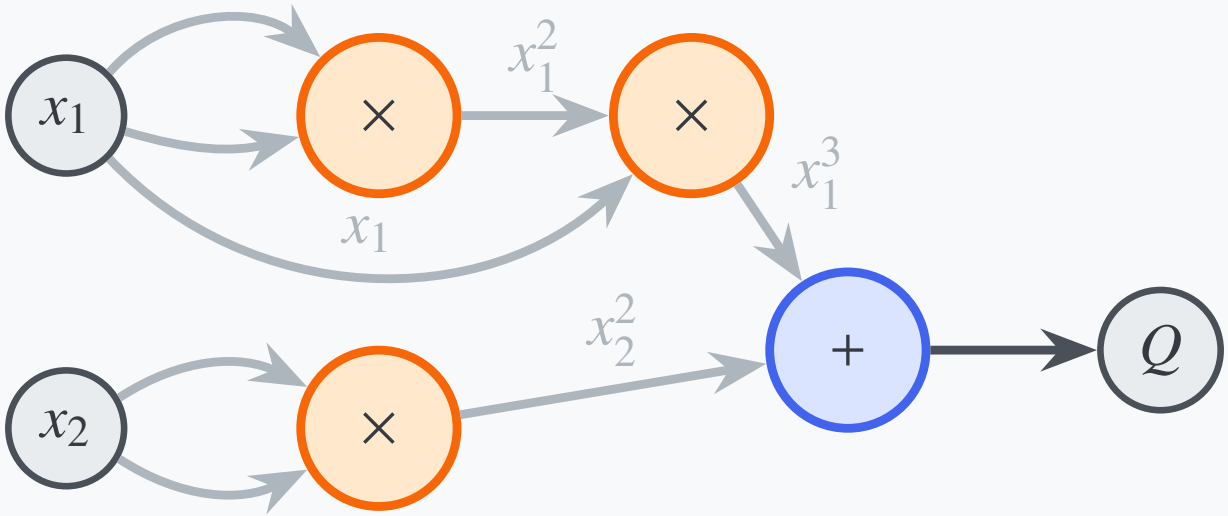


Figure: Example of a circuit evaluating $x_1^3 + x_2^2$.

Arithmetic Circuits Example III

Example

Well, it is quite clear how to represent any polynomial-like expressions. But how can we translate `if` statements?

```
def example(a: bool, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

Arithmetic Circuits Example III

Example

Well, it is quite clear how to represent any polynomial-like expressions. But how can we translate `if` statements?

```
def example(a: bool, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

We can transform such a function into the next expression:

$$r = a \times (b \times c) + (1 - a) \times (b + c)$$

Arithmetic Circuits Example III

Example

Well, it is quite clear how to represent any polynomial-like expressions. But how can we translate `if` statements?

```
def example(a: bool, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

We can transform such a function into the next expression:

$$r = a \times (b \times c) + (1 - a) \times (b + c)$$

Corresponding equations for the circuit are:

$$r_1 = b \times c, \qquad r_3 = 1 - a, \qquad r_5 = r_3 \times r_2$$
$$r_2 = b + c, \qquad r_4 = a \times r_1, \qquad r = r_4 + r_5$$

Arithmetic Circuits Example III

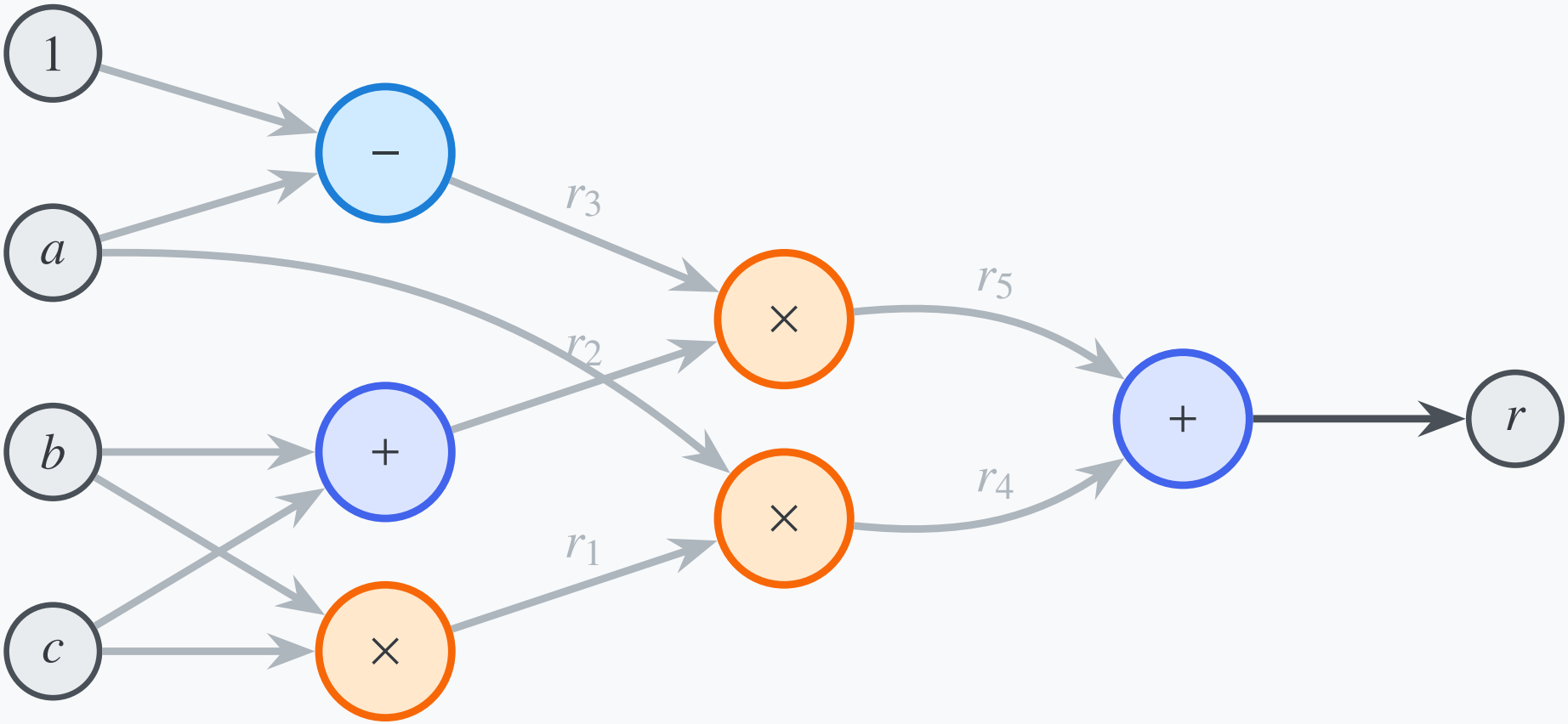


Figure: Example of a circuit evaluating the `if` statement logic.

Linear Algebra Recap

Linear Algebra Recap

We will need a few operations from the Linear Algebra lecture (Lecture 3). For $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$ and matrices A, B:

- **Matrix-vector product:** $A\mathbf{x}$ for $A \in \mathbb{F}^{m \times n}$, $\mathbf{x} \in \mathbb{F}^n$; vectors are columns, $\mathbf{x}^\top$ is a row.

Linear Algebra Recap

We will need a few operations from the Linear Algebra lecture (Lecture 3). For $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$ and matrices A, B:

- **Matrix-vector product:** $A\mathbf{x}$ for $A \in \mathbb{F}^{m \times n}$, $\mathbf{x} \in \mathbb{F}^n$; vectors are columns, $\mathbf{x}^\top$ is a row.
- **Dot (inner) product:** $\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$.

Linear Algebra Recap

We will need a few operations from the Linear Algebra lecture (Lecture 3). For $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$ and matrices A, B:

- **Matrix-vector product:** $A\mathbf{x}$ for $A \in \mathbb{F}^{m \times n}$, $\mathbf{x} \in \mathbb{F}^n$; vectors are columns, $\mathbf{x}^\top$ is a row.
- **Dot (inner) product:** $\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$.
- **Hadamard product:** $(A \odot B)_{i,j} = A_{i,j} B_{i,j}$ (element-wise).

Linear Algebra Recap

We will need a few operations from the Linear Algebra lecture (Lecture 3). For $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$ and matrices A, B:

- **Matrix-vector product:** $A\mathbf{x}$ for $A \in \mathbb{F}^{m \times n}$, $\mathbf{x} \in \mathbb{F}^n$; vectors are columns, $\mathbf{x}^\top$ is a row.
- **Dot (inner) product:** $\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$.
- **Hadamard product:** $(A \odot B)_{i,j} = A_{i,j} B_{i,j}$ (element-wise).
- **Outer product:** $\mathbf{u} \otimes \mathbf{v} := \mathbf{u}\mathbf{v}^\top$, a rank-1 matrix for non-zero $\mathbf{u}, \mathbf{v}$.

Linear Algebra Recap

We will need a few operations from the Linear Algebra lecture (Lecture 3). For $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$ and matrices A, B:

- **Matrix-vector product:** $A\mathbf{x}$ for $A \in \mathbb{F}^{m \times n}$, $\mathbf{x} \in \mathbb{F}^n$; vectors are columns, $\mathbf{x}^\top$ is a row.
- **Dot (inner) product:** $\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$.
- **Hadamard product:** $(A \odot B)_{i,j} = A_{i,j} B_{i,j}$ (element-wise).
- **Outer product:** $\mathbf{u} \otimes \mathbf{v} := \mathbf{u}\mathbf{v}^\top$, a rank-1 matrix for non-zero $\mathbf{u}, \mathbf{v}$.

Example

For $\mathbf{u} = (1, 2, 3)$ and $\mathbf{v} = (2, 4, 3)$: $\langle \mathbf{u}, \mathbf{v} \rangle = 2 + 8 + 9 = 19$ and $\mathbf{u} \odot \mathbf{v} = (2, 8, 9)$.

Rank-1 Constraint System

Constraint Definition

Definition

Each **constraint** in the Rank-1 Constraint System must be in the form:

$$\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$$

Where $\mathbf{z}$ is a vector containing all the *input*, *output*, and *intermediate* variables involved in the computation. The vectors $\boldsymbol{\ell}$, $\boldsymbol{r}$, and $\boldsymbol{o}$ are vectors of coefficients corresponding to these variables, and they define the relationship between the linear combinations of $\mathbf{z}$ on the left-hand side and the right-hand side of the equation.

Constraint Example

Example

Consider the most basic circuit with one multiplication gate: $x_1 \times x_2 = r$. The witness vector $\mathbf{z} = (r, x_1, x_2)$. So

$$z_2 \times z_3 = z_1$$
$$(0 + z_2 + 0) \times (0 + 0 + z_3) = z_1 + 0 + 0$$
$$(0z_1 + 1z_2 + 0z_3) \times (0z_1 + 0z_2 + 1z_3) = 1z_1 + 0z_2 + 0z_3$$

Therefore the coefficients vectors are:

$$\boldsymbol{\ell} = (0, 1, 0), \quad \boldsymbol{r} = (0, 0, 1), \quad \boldsymbol{o} = (1, 0, 0).$$

The general form of our constraint is:

$$(\ell_1 z_1 + \ell_2 z_2 + \ell_3 z_3)(r_1 z_1 + r_2 z_2 + r_3 z_3) = o_1 z_1 + o_2 z_2 + o_3 z_3$$

Constraint System Example

Now, let us consider a more complex example.

```
def r(x1: F, x2: F, x3: F) -> F:
    return x2 * x3 if x1 else x2 + x3
```

That can be expressed as:

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Constraint System Example

Now, let us consider a more complex example.

```
def r(x1: F, x2: F, x3: F) -> F:
    return x2 * x3 if x1 else x2 + x3
```

That can be expressed as:

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Note

We need a boolean restriction for x_1: that is, $x_1 \times (1 - x_1) = 0$.

Constraint System Example

Now, let us consider a more complex example.

```
def r(x1: F, x2: F, x3: F) -> F:
    return x2 * x3 if x1 else x2 + x3
```

That can be expressed as:

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Note

We need a boolean restriction for x_1: that is, $x_1 \times (1 - x_1) = 0$.

Thus, the next constraints can be built:

$$x_1 \times x_1 = x_1 \quad \text{(binary check)} \tag{1}$$

$$x_2 \times x_3 = \text{mult} \tag{2}$$

$$x_1 \times \text{mult} = \text{selectMult} \tag{3}$$

$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \tag{4}$$

Constraint System Example

The witness vector: $\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$. The coefficients vectors:

$$\begin{aligned}
\boldsymbol{\ell}_1 &= (0,0,1,0,0,0,0), & \boldsymbol{r}_1 &= (0,0,1,0,0,0,0), & \boldsymbol{o}_1 &= (0,0,1,0,0,0,0)\\
\boldsymbol{\ell}_2 &= (0,0,0,1,0,0,0), & \boldsymbol{r}_2 &= (0,0,0,0,1,0,0), & \boldsymbol{o}_2 &= (0,0,0,0,0,1,0)\\
\boldsymbol{\ell}_3 &= (0,0,1,0,0,0,0), & \boldsymbol{r}_3 &= (0,0,0,0,0,1,0), & \boldsymbol{o}_3 &= (0,0,0,0,0,0,1)\\
\boldsymbol{\ell}_4 &= (1,0,-1,0,0,0,0), & \boldsymbol{r}_4 &= (0,0,0,1,1,0,0), & \boldsymbol{o}_4 &= (0,1,0,0,0,0,-1)
\end{aligned}$$

Using the arithmetic in a large $\mathbb{F}_p$, consider the following values:

$$x_1 = 1, \quad x_2 = 3, \quad x_3 = 4$$

Verifying the constraints:

1. $x_1 \times x_1 = x_1 \quad (1 \times 1 = 1)$
2. $x_2 \times x_3 = \text{mult} \quad (3 \times 4 = 12)$
3. $x_1 \times \text{mult} = \text{selectMult} \quad (1 \times 12 = 12)$
4. $(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \quad (0 \times 7 = 12 - 12)$

R1CS In Matrix Form

Theorem

Consider a **Rank-1 Constraint System** (**R1CS**) defined over m constraints. Each constraint in such system is associated with coefficient vectors $\boldsymbol{\ell}_i$, $\boldsymbol{r}_i$, and $\boldsymbol{o}_i$, where $i \in \{1, 2, \ldots, m\}$ and also a witness vector $\mathbf{z}$ consisting of n elements.
Then this system can also be represented using the corresponding matrices L, R, and O.

$$L = \begin{bmatrix} \ell_{1,1} & \ell_{1,2} & \cdots & \ell_{1,n} \\ \ell_{2,1} & \ell_{2,2} & \cdots & \ell_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{m,1} & \ell_{m,2} & \cdots & \ell_{m,n} \end{bmatrix}, \text{ same for } R \text{ and } O,$$

such that all constraints can be reduced to the single equation:

$$L\mathbf{z} \odot R\mathbf{z} = O\mathbf{z}$$

R1CS In Matrix Form

Proof. Matrices defined this way can be expressed as

$$L = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix}, \quad R = \begin{bmatrix} \boldsymbol{r}_1^\top \\ \boldsymbol{r}_2^\top \\ \vdots \\ \boldsymbol{r}_m^\top \end{bmatrix}, \quad O = \begin{bmatrix} \boldsymbol{o}_1^\top \\ \boldsymbol{o}_2^\top \\ \vdots \\ \boldsymbol{o}_m^\top \end{bmatrix}$$

R1CS In Matrix Form

Proof. Matrices defined this way can be expressed as

$$L = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix}, \quad R = \begin{bmatrix} \boldsymbol{r}_1^\top \\ \boldsymbol{r}_2^\top \\ \vdots \\ \boldsymbol{r}_m^\top \end{bmatrix}, \quad O = \begin{bmatrix} \boldsymbol{o}_1^\top \\ \boldsymbol{o}_2^\top \\ \vdots \\ \boldsymbol{o}_m^\top \end{bmatrix}$$

Consider an expression $L\mathbf{z}$:

$$L\mathbf{z} = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{\ell}_1^\top \mathbf{z} \\ \boldsymbol{\ell}_2^\top \mathbf{z} \\ \vdots \\ \boldsymbol{\ell}_m^\top \mathbf{z} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n \ell_{1,i} z_i \\ \sum_{i=1}^n \ell_{2,i} z_i \\ \vdots \\ \sum_{i=1}^n \ell_{m,i} z_i \end{bmatrix} = \begin{bmatrix} \langle \boldsymbol{\ell}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{\ell}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{\ell}_m, \mathbf{z} \rangle \end{bmatrix}$$

R1CS In Matrix Form

Proof. Matrices defined this way can be expressed as

$$L = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix}, \quad R = \begin{bmatrix} \boldsymbol{r}_1^\top \\ \boldsymbol{r}_2^\top \\ \vdots \\ \boldsymbol{r}_m^\top \end{bmatrix}, \quad O = \begin{bmatrix} \boldsymbol{o}_1^\top \\ \boldsymbol{o}_2^\top \\ \vdots \\ \boldsymbol{o}_m^\top \end{bmatrix}$$

Consider an expression $L\mathbf{z}$:

$$L\mathbf{z} = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{\ell}_1^\top \mathbf{z} \\ \boldsymbol{\ell}_2^\top \mathbf{z} \\ \vdots \\ \boldsymbol{\ell}_m^\top \mathbf{z} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n \ell_{1,i} z_i \\ \sum_{i=1}^n \ell_{2,i} z_i \\ \vdots \\ \sum_{i=1}^n \ell_{m,i} z_i \end{bmatrix} = \begin{bmatrix} \langle \boldsymbol{\ell}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{\ell}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{\ell}_m, \mathbf{z} \rangle \end{bmatrix}$$

Therefore, we have:

$$L\mathbf{z} = \begin{bmatrix} \langle \boldsymbol{\ell}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{\ell}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{\ell}_m, \mathbf{z} \rangle \end{bmatrix}, \quad R\mathbf{z} = \begin{bmatrix} \langle \boldsymbol{r}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{r}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{r}_m, \mathbf{z} \rangle \end{bmatrix}, \quad O\mathbf{z} = \begin{bmatrix} \langle \boldsymbol{o}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{o}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{o}_m, \mathbf{z} \rangle \end{bmatrix}$$

R1CS In Matrix Form

Proof. Matrices defined this way can be expressed as

$$L = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix}, \quad R = \begin{bmatrix} \boldsymbol{r}_1^\top \\ \boldsymbol{r}_2^\top \\ \vdots \\ \boldsymbol{r}_m^\top \end{bmatrix}, \quad O = \begin{bmatrix} \boldsymbol{o}_1^\top \\ \boldsymbol{o}_2^\top \\ \vdots \\ \boldsymbol{o}_m^\top \end{bmatrix}$$

Consider an expression $L\mathbf{z}$:

$$L\mathbf{z} = \begin{bmatrix} \boldsymbol{\ell}_1^\top \\ \boldsymbol{\ell}_2^\top \\ \vdots \\ \boldsymbol{\ell}_m^\top \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{\ell}_1^\top \mathbf{z} \\ \boldsymbol{\ell}_2^\top \mathbf{z} \\ \vdots \\ \boldsymbol{\ell}_m^\top \mathbf{z} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n \ell_{1,i} z_i \\ \sum_{i=1}^n \ell_{2,i} z_i \\ \vdots \\ \sum_{i=1}^n \ell_{m,i} z_i \end{bmatrix} = \begin{bmatrix} \langle \boldsymbol{\ell}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{\ell}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{\ell}_m, \mathbf{z} \rangle \end{bmatrix}$$

Therefore, we have:

$$L\mathbf{z} = \begin{bmatrix} \langle \boldsymbol{\ell}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{\ell}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{\ell}_m, \mathbf{z} \rangle \end{bmatrix}, \quad R\mathbf{z} = \begin{bmatrix} \langle \boldsymbol{r}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{r}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{r}_m, \mathbf{z} \rangle \end{bmatrix}, \quad O\mathbf{z} = \begin{bmatrix} \langle \boldsymbol{o}_1, \mathbf{z} \rangle \\ \langle \boldsymbol{o}_2, \mathbf{z} \rangle \\ \vdots \\ \langle \boldsymbol{o}_m, \mathbf{z} \rangle \end{bmatrix}$$

Thus, $L\mathbf{z} \odot R\mathbf{z} = O\mathbf{z}$ is equivalent to the system of m constraints:

$$\langle \boldsymbol{\ell}_j, \mathbf{z} \rangle \times \langle \boldsymbol{r}_j, \mathbf{z} \rangle = \langle \boldsymbol{o}_j, \mathbf{z} \rangle, \; j \in \{1, \ldots, m\}.$$

R1CS In Matrix Form

Example

The vectors $\boldsymbol{\ell}_i$ from the previous examples have the form:

$$\boldsymbol{\ell}_1 = (0, 0, 1, 0, 0, 0, 0)$$
$$\boldsymbol{\ell}_2 = (0, 0, 0, 1, 0, 0, 0)$$
$$\boldsymbol{\ell}_3 = (0, 0, 1, 0, 0, 0, 0)$$
$$\boldsymbol{\ell}_4 = (1, 0, -1, 0, 0, 0, 0)$$

This corresponds to $n = 7, m = 4$, so the matrix L becomes:

$$\begin{bmatrix} \ell_{1,1} & \ell_{1,2} & \ell_{1,3} & \ell_{1,4} & \ell_{1,5} & \ell_{1,6} & \ell_{1,7} \\ \ell_{2,1} & \ell_{2,2} & \ell_{2,3} & \ell_{2,4} & \ell_{2,5} & \ell_{2,6} & \ell_{2,7} \\ \ell_{3,1} & \ell_{3,2} & \ell_{3,3} & \ell_{3,4} & \ell_{3,5} & \ell_{3,6} & \ell_{3,7} \\ \ell_{4,1} & \ell_{4,2} & \ell_{4,3} & \ell_{4,4} & \ell_{4,5} & \ell_{4,6} & \ell_{4,7} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Why Rank-1?

Lemma

Suppose we have a constraint $\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$ with coefficient vectors $\boldsymbol{\ell}$, $\boldsymbol{r}$, $\boldsymbol{o}$ and witness vector $\mathbf{z}$ (all from $\mathbb{F}^n$). Then it can be expressed in the form:

$$\mathbf{z}^\top T \mathbf{z} - \boldsymbol{o}^\top \mathbf{z} = 0$$

Where $T = \boldsymbol{\ell} \otimes \boldsymbol{r}$ — **rank-1** matrix.

Why Rank-1?

Lemma

Suppose we have a constraint $\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$ with coefficient vectors $\boldsymbol{\ell}$, $\boldsymbol{r}$, $\boldsymbol{o}$ and witness vector $\mathbf{z}$ (all from $\mathbb{F}^n$). Then it can be expressed in the form:

$$\mathbf{z}^\top T \mathbf{z} - \boldsymbol{o}^\top \mathbf{z} = 0$$

Where $T = \boldsymbol{\ell} \otimes \boldsymbol{r}$ — **rank-1** matrix.

Lemma proof. Consider $\boldsymbol{\ell}, \boldsymbol{r}, \boldsymbol{o}, \mathbf{z} \in \mathbb{F}^n$.

$$\left(\sum_{i=1}^{n} \ell_i z_i \right) \times \left(\sum_{j=1}^{n} r_j z_j \right) = \sum_{k=1}^{n} o_k z_k$$

Combine the products into a double sum on the left side:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} \ell_i r_j z_i z_j = \mathbf{z}^\top (\boldsymbol{\ell} \otimes \boldsymbol{r}) \mathbf{z} = \mathbf{z}^\top T \mathbf{z}$$

Thus, the constraint can be written as $\mathbf{z}^\top T \mathbf{z} - \boldsymbol{o}^\top \mathbf{z} = 0$.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream